

SMART PARKING LOT ALLOCATION AND BILLING SYSTEM

DBMS Course Project — Final Report

Name	Saranjyot Singh Jassal
Roll Number	[ENTER YOUR ROLL NUMBER]
Course	DBMS
Faculty	[ENTER FACULTY NAME]
Academic Year	2026

2. Abstract

The Smart Parking Lot Allocation and Billing System is a relational database project designed to manage multiple parking facilities, individual parking slots, registered vehicles, parking sessions, pricing, payments and staff accountability. Stored procedures automate slot allocation and billing, while triggers maintain slot occupancy. Reporting views provide occupancy, active-session and revenue information.

3. Introduction and Problem Statement

Manual parking management can cause delays, inaccurate occupancy information and inconsistent billing. A centralized DBMS can maintain parking infrastructure and transaction records while enforcing relationships and constraints. The project models the complete parking lifecycle from vehicle entry and slot assignment to exit, billing and release of the slot.

4. Objectives and Scope

Objectives include supporting multiple parking lots; allocating free slots according to vehicle type; tracking entry and exit; calculating charges using lot-specific rate cards and grace periods; recording payment information and staff accountability; and providing reporting views. The scope is a database-backed demonstration system rather than a production IoT parking deployment.

5. Software and Hardware Requirements

Software: MySQL/MariaDB, Python 3, Tkinter and mysql-connector-python. Development can be performed on a normal Windows/Linux/macOS computer. Hardware requirements are modest: a computer capable of running MySQL and Python, with sufficient RAM and storage for the project database.

6. ER Diagram

The final schema contains ParkingLot, ParkingSlot, RateCard, Vehicle, ParkingSession, Payment and Staff. ParkingLot is the parent entity for slots, rate cards and staff. Vehicles participate in parking sessions; sessions use slots and generate payments.

7. Relational Schema and Normalization

The design separates parking infrastructure, vehicle master data, pricing, sessions, payments and staff into separate relations. Primary keys uniquely identify rows and foreign keys enforce referential integrity. The decomposition reduces repeating groups and update anomalies and is suitable for a normalized transactional schema.

8. Data Dictionary

ParkingLot: lot_id PK, lot_name, location, total_slots. ParkingSlot: slot_id PK, lot_id FK, slot_number, slot_type, status. Vehicle: vehicle_id PK, plate_number UNIQUE, vehicle_type, owner_name, phone. RateCard: rate_id PK, lot_id FK, vehicle_type, rate_per_hour, grace_minutes. ParkingSession: session_id PK, vehicle_id FK, slot_id FK, entry_time, exit_time, status, created_at. Payment: payment_id PK, session_id FK/UNIQUE, duration_mins, amount, payment_mode, payment_status, staff_id FK, payment_time, created_at. Staff: staff_id PK, full_name, role, lot_id FK.

9. SQL Commands Used

DDL commands create the database, tables, keys, indexes, triggers, procedures and views. DML commands insert master data and transaction records. SELECT statements verify data and reporting views. The complete executable SQL is provided in Presentation-II/smart_parking_final.sql.

10. Queries with Outputs

The project includes SELECT queries for parking lots, slots, vehicles, rates, staff and reporting views. Procedure calls demonstrate allocation and billing. The exact Presentation-II faculty-assigned query must be inserted into query_solution_Presentation-II.md because it was not included in the supplied project materials.

11. UI Design and Screenshots

Presentation-III contains a Python Tkinter UI connected to MySQL. It supports insertion, deletion and viewing of Vehicle records. The required screenshots should be captured by the student while running the UI: before insertion, after insertion, before deletion, after deletion, and the view screen.

12. Implementation Details

The database uses MySQL/MariaDB. Stored procedures implement allocation and billing logic. Triggers synchronize slot status with parking-session state. Python Tkinter provides the desktop UI and mysql-connector-python provides DB connectivity. The source code is in Presentation-III/app.py.

13. Testing

Test cases should verify: inserting a valid vehicle; viewing records; deleting an unreferenced vehicle; rejecting duplicate plate numbers; allocating a matching free slot; handling no available slots; calculating a bill after simulated parking time; and automatically releasing a completed slot. Record actual results and screenshots during the final demonstration.

14. Conclusion and Future Enhancements

The system demonstrates how a relational DBMS can automate core parking operations while preserving consistency through keys, constraints, procedures and triggers. Future enhancements could include a web/mobile interface, real-time sensor integration, online payment gateway integration, authentication and role-based access, reservations and analytics.

15. References

MySQL/MariaDB documentation and course materials used for database design, SQL, constraints, stored procedures, triggers and relational modeling.

16. Appendix — GitHub Repository Link

Repository: [https://github.com/\[YOUR-USERNAME\]/DBMS-Course-Project](https://github.com/[YOUR-USERNAME]/DBMS-Course-Project)

ER Diagram

Database Design — Final Schema

Smart Parking Lot Allocation and Billing System (Updated)

